

■ Java 8 + DSA + System Design

Complete Interview Preparation Guide

Foundation → Advanced

Easy → Hard

Core → Design

60+ Interview Questions with Detailed Answers

Java 8 • Data Structures & Algorithms • System Design

■ JAVA 8 — Foundation to Advanced

Q: What are Lambda Expressions and why were they introduced?

Lambda expressions are anonymous functions — blocks of code that can be passed around as values. They were introduced in Java 8 to bring functional programming capabilities into Java, enabling cleaner, more concise code especially when working with collections and streams.

Before Java 8, passing behaviour required anonymous inner classes which were verbose. Lambda expressions solve this:

```
// Before Java 8 (Anonymous Inner Class) Runnable r = new Runnable() { public void run() {
System.out.println("Hello"); } }; // Java 8 Lambda Runnable r = () ->
System.out.println("Hello");
```

Syntax: (parameters) -> expression or (parameters) -> { statements; }

✓ *Key Benefit: Enables treating functions as first-class citizens — pass behaviour as method arguments.*

Q: What is Functional Interface? Explain with example.

A Functional Interface is an interface with exactly ONE abstract method. It can have multiple default/static methods but only one abstract method. The `@FunctionalInterface` annotation is used to enforce this contract at compile time.

```
@FunctionalInterface interface Calculator { int calculate(int a, int b); // single
abstract method } // Usage with Lambda Calculator add = (a, b) -> a + b; Calculator mul =
(a, b) -> a * b; System.out.println(add.calculate(3, 4)); // 7
```

Built-in functional interfaces in Java 8:

- Predicate — takes T, returns boolean → `test()`
- Function — takes T, returns R → `apply()`
- Consumer — takes T, returns void → `accept()`
- Supplier — takes nothing, returns T → `get()`
- BiFunction — takes two args, returns R

Q: Difference between Stream API and Collection

Feature	Collection	Stream
Purpose	Store & manage data	Process & compute data
Data storage	Stores elements	Does not store data
Traversal	Can traverse multiple times	Can be traversed only once
Modification	Can add/remove elements	Cannot modify source
Evaluation	Eager (immediate)	Lazy (on terminal op)
External iteration	for/foreach loop	Internal iteration
Parallel processing	Manual (complex)	Easy via <code>parallelStream()</code>

Q: How does `map()` vs `flatMap()` work?

`map()` transforms each element into exactly one result — one-to-one mapping. `flatMap()` transforms each element into zero or more results and flattens them into a single stream — one-to-many mapping.

```
// map() – wraps each word in a Stream List words = List.of("Hello", "World");
words.stream().map(String::toUpperCase) // Stream .forEach(System.out::println); //
HELLO WORLD // flatMap() – flattens Stream into Stream List sentences = List.of("Hi
There", "Good Morning"); sentences.stream().flatMap(s -> Arrays.stream(s.split(" ")))
.forEach(System.out::println); // Hi There Good Morning
```

✓ *Rule of thumb: Use map() for 1-to-1 transformations. Use flatMap() when each element expands to a list/array (e.g., splitting sentences into words, flattening nested lists).*

Q: What is Optional and how does it prevent NullPointerException?

Optional is a container object introduced in Java 8 to represent a value that may or may not be present. It forces the developer to explicitly handle the absence of a value, eliminating NullPointerExceptions caused by forgotten null checks.

```
Optional name = Optional.ofNullable(getUserName()); // Safe retrieval String result =
name.orElse("Anonymous"); // default value String result = name.orElseGet(() ->
fetchDefault()); // lazy default name.ifPresent(n -> System.out.println("Hello " + n));
// only if present // Chain operations Optional len = name.map(String::length); // Throw
exception if absent String val = name.orElseThrow(() -> new RuntimeException("No
name!"));
```

■ *Never call optional.get() without isPresent() check — defeats the purpose. Prefer orElse/orElseGet/ifPresent.*

Q: Difference between forEach() and peek()

forEach() is a terminal operation — it consumes the stream and is used for side effects (printing, saving). peek() is an intermediate operation — it allows you to inspect elements mid-pipeline without consuming the stream.

```
// peek() – intermediate, used for debugging List result = list.stream().peek(e ->
System.out.println("Before: " + e)).filter(e -> e.startsWith("A")).peek(e ->
System.out.println("After filter: " + e)).collect(Collectors.toList()); // forEach() –
terminal, ends the stream list.stream().forEach(System.out::println);
```

✓ *peek() is primarily used for debugging. forEach() is for final consumption/side effects.*

Q: What are Method References? Types?

Method references are a shorthand for lambdas that simply call an existing method. Syntax: ClassName::methodName

Type	Syntax	Equivalent Lambda
Static method	ClassName::staticMethod	x -> ClassName.staticMethod(x)
Instance method (specific)	instance::method	x -> instance.method(x)
Instance method (arbitrary)	ClassName::instanceMethod	(obj, x) -> obj.method(x)
Constructor reference	ClassName::new	() -> new ClassName()

```
list.stream().forEach(System.out::println); // instance method ref
list.stream().map(String::toUpperCase); // arbitrary instance method
list.stream().map(Person::new); // constructor reference
```

Q: How does filter() work in Streams?

filter() is an intermediate operation that takes a Predicate and returns a new stream containing only elements that match the predicate. It does not modify the original collection.

```
List nums = List.of(1, 2, 3, 4, 5, 6, 7, 8); List evens = nums.stream().filter(n -> n % 2 == 0) // keep only even numbers .collect(Collectors.toList()); // [2, 4, 6, 8] //
Chaining multiple filters List result = employees.stream().filter(e -> e.getDept().equals("Engineering")).filter(e -> e.getSalary() > 80000)
.map(Employee::getName) .collect(Collectors.toList());
```

✓ *filter() is lazy — it won't process elements until a terminal operation (collect, count, findFirst) is called.*

Q: What is Parallel Stream and when NOT to use it?

Parallel streams split the data into multiple chunks and process them simultaneously using the ForkJoinPool (default: all available CPU cores). Simply replace .stream() with .parallelStream().

```
long count = largeList.parallelStream().filter(n -> n % 2 == 0) .count(); // processes
on multiple threads
```

When NOT to use parallel streams:

- Small datasets — thread overhead costs more than the gain
- Ordered operations where sequence matters (findFirst on parallel may be non-deterministic)
- Operations with shared mutable state — risk of race conditions
- I/O bound tasks — parallelism helps only for CPU-bound work
- When using non-thread-safe data structures

■ *Rule: Benchmark before using. Parallel streams can actually be SLOWER for small lists due to overhead.*

Q: What is the difference between findFirst() and findAny()?

Both return an Optional with one element from a stream. The difference matters only in parallel streams:

- findFirst() — always returns the FIRST element in encounter order. Safe in sequential & parallel, but parallel must coordinate threads to guarantee order (slower).
- findAny() — returns ANY element, whichever thread finds one first. Non-deterministic in parallel streams but FASTER.

```
// Sequential — both return first matching element Optional first =
list.stream().filter(s -> s.startsWith("A")).findFirst(); Optional any =
list.stream().filter(s -> s.startsWith("A")).findAny(); // Parallel — findAny() is faster
(no ordering guarantee) Optional fast = list.parallelStream().filter(pred).findAny();
```

✓ *Use findFirst() when order matters. Use findAny() in parallel streams for maximum performance.*

■ DATA STRUCTURES & ALGORITHMS — Easy to Medium

Q: What is the Sliding Window technique?

Sliding Window is a technique for efficiently solving problems involving contiguous subarrays or substrings. Instead of re-computing for every window from scratch ($O(n*k)$), we slide a window across the array, adding the new element and removing the old one — achieving $O(n)$.

```
// Max sum of subarray of size k
public int maxSum(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    maxSum = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k]; // slide the window
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

✓ Used for: Maximum/minimum subarray, Longest substring without repeating chars, Smallest subarray with sum $\geq k$.

Q: How does Two Pointer approach work?

Two Pointers uses two indices (usually left and right) moving towards each other or in the same direction to solve problems in $O(n)$ instead of $O(n^2)$. Works best on sorted arrays.

```
// Two Sum in sorted array
public int[] twoSum(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++; // need larger sum
        else right--; // need smaller sum
    }
    return new int[]{-1, -1};
}
```

✓ Used for: Pair with target sum, Remove duplicates, Container with most water, 3-Sum problem.

Q: How to detect a cycle in a Linked List?

Floyd's Cycle Detection (Tortoise and Hare): Use two pointers — slow moves 1 step, fast moves 2 steps. If they meet, there's a cycle. If fast reaches null, no cycle. Time: $O(n)$, Space: $O(1)$.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; // 1 step
        fast = fast.next.next; // 2 steps
        if (slow == fast) return true; // cycle detected!
    }
    return false;
}
```

✓ To find the START of the cycle: reset slow to head, move both one step at a time — they meet at the cycle start.

Q: Difference between BFS and DFS

Feature	BFS	DFS
Data structure	Queue (FIFO)	Stack / Recursion
Traversal order	Level by level	Depth first (branch by branch)
Space complexity	$O(w)$ — width of tree	$O(h)$ — height of tree
Shortest path	Yes (unweighted graph)	Not guaranteed
Best for	Shortest path, Level order	Cycle detection, Topological sort
When graph is wide	Poor (large queue)	Better
When graph is deep	Better	May stack overflow

Q: How to solve Two Sum using HashMap?

Store each number's index in a HashMap. For each element, check if (target - element) already exists in the map. This gives $O(n)$ time and $O(n)$ space.

```
public int[] twoSum(int[] nums, int target) { Map map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int complement = target - nums[i]; if (map.containsKey(complement)) { return new int[]{map.get(complement), i}; } map.put(nums[i], i); // store number -> index } return new int[]{}; // no solution } // Time: O(n) Space: O(n)
```

Q: What is Divide and Conquer strategy?

Divide and Conquer breaks a problem into smaller subproblems, solves each recursively, and combines the results. Three steps: Divide → Conquer (recurse) → Combine.

- Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication
- Merge Sort: Divide array in half → sort each half → merge sorted halves. $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$

✓ *Key insight: Use when a problem can be broken into independent subproblems of the same type.*

Q: What are Greedy Algorithms and their limitations?

Greedy algorithms make the locally optimal choice at each step, hoping it leads to a globally optimal solution. They never backtrack — once a choice is made, it stays.

- Examples: Activity Selection, Huffman Coding, Dijkstra's (greedy for shortest path), Kruskal's MST
- Limitations: Does NOT always give the globally optimal solution. E.g., Greedy fails for 0/1 Knapsack (use DP instead).

```
// Greedy – Coin change (works only with canonical coin systems) // For coins [1,5,10,25] and amount=36: 25+10+1 = 3 coins // But for coins [1,3,4] and amount=6: Greedy gives 4+1+1=3, DP gives 3+3=2
```

■ *Greedy works when the problem has 'greedy choice property' and 'optimal substructure'. Always verify before applying.*

Q: What is a Trie and where is it used?

A Trie (prefix tree) is a tree data structure where each node represents a character. It efficiently stores and retrieves strings based on prefixes. Insert/Search: $O(L)$ where L = word length.

```
class TrieNode { TrieNode[] children = new TrieNode[26]; boolean isEnd = false; } // Search 'apple': root -> 'a' -> 'p' -> 'p' -> 'l' -> 'e' (isEnd=true)
```

- Autocomplete / search suggestions (Google search bar)
- Spell checkers — find words with common prefix
- IP routing — longest prefix matching
- Word games — finding valid words

Q: How to find Kth Largest Element using Heap?

Use a Min-Heap of size K . Iterate through all elements — if element is larger than heap's minimum, replace it. The heap top is the K th largest. Time: $O(n \log k)$, Space: $O(k)$.

```
public int findKthLargest(int[] nums, int k) { PriorityQueue minHeap = new PriorityQueue<>(); // default min-heap for (int num : nums) { minHeap.offer(num); if (minHeap.size() > k) minHeap.poll(); // remove smallest } return minHeap.peek(); // top of heap = kth largest }
```

✓ *Alternative: QuickSelect gives $O(n)$ average time. Use heap for streaming data (online algorithm).*

Q: Difference between Merge Sort and Quick Sort

Feature	Merge Sort	Quick Sort
Strategy	Divide & Conquer + Merge	Divide & Conquer + Partition
Time (avg)	$O(n \log n)$	$O(n \log n)$
Time (worst)	$O(n \log n)$	$O(n^2)$ — bad pivot
Space	$O(n)$ — needs extra array	$O(\log n)$ — in-place
Stable sort	Yes	No (basic version)
Best for	Linked lists, stable sort needed	Arrays, cache-friendly
Preferred in	Java Arrays.sort (objects)	Primitive arrays

■ DATA STRUCTURES & ALGORITHMS — Medium to Hard

Q: How to solve 0/1 Knapsack using Dynamic Programming?

For each item, decide: include it (if weight fits) or exclude it. Use a 2D DP table where $dp[i][w]$ = max value using first i items with capacity w .

```
// items: weight[] and value[], capacity W
public int knapsack(int[] wt, int[] val, int W) {
    int n = wt.length;
    int[][] dp = new int[n+1][W+1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w]; // exclude item i
            if (wt[i-1] <= w) // include item i if it fits
                dp[i][w] = Math.max(dp[i][w], val[i-1] + dp[i-1][w-wt[i-1]]);
        }
    }
    return dp[n][W];
} // Time: O(nW) Space: O(nW)
```

✓ *Space optimization: Use 1D dp array, iterate weights from right to left.*

Q: What is Backtracking and when to use it?

Backtracking is a refined brute-force approach that incrementally builds solutions and abandons ('backtracks') a path as soon as it determines this path cannot lead to a valid solution. It explores a decision tree.

```
// N-Queens: place queens so none attack each other
void solve(int row, boolean[] cols, boolean[] diag1, boolean[] diag2) {
    if (row == n) { result.add(buildBoard()); return; }
    for (int col = 0; col < n; col++) {
        if (cols[col] || diag1[row-col+n] || diag2[row+col]) continue;
        cols[col] = diag1[row-col+n] = diag2[row+col] = true;
        solve(row+1, cols, diag1, diag2);
        cols[col] = diag1[row-col+n] = diag2[row+col] = false; // backtrack!
    }
}
```

✓ *Use when: Generating all permutations/combinations, Sudoku solver, N-Queens, Word search, Subset sum.*

Q: Binary Search in Rotated Arrays

A sorted array rotated at a pivot still has the property that one half is always sorted. Modified binary search: identify which half is sorted, check if target lies in that half.

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[lo] <= nums[mid]) { // left half is sorted
            if (target >= nums[lo] && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right half is sorted
            if (target > nums[mid] && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
} // Time: O(log n)
```

Q: Detect Islands in Matrix using DFS

Count connected components of '1's in a 2D grid. Use DFS from each unvisited '1' to mark all connected land cells as visited (sink them to '0').

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++)
        for (int j = 0; j < grid[0].length; j++)
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
    return count;
}

void dfs(char[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') return;
    grid[r][c] = '0';
    dfs(grid, r+1, c);
    dfs(grid, r-1, c);
    dfs(grid, r, c+1);
    dfs(grid, r, c-1);
} // Time: O(M*N)
```

Q: Bit Manipulation for XOR Problems

XOR (^) has key properties: $a \wedge a = 0$, $a \wedge 0 = a$, XOR is commutative and associative. These make it powerful for finding unique/missing numbers.


```
// Find single number (all others appear twice) int single = 0; for (int n : nums) single
^= n; // pairs cancel out // Find two non-repeating numbers int xorAll = 0; for (int n :
nums) xorAll ^= n; // xor of two unique numbers int bit = xorAll & (-xorAll); // rightmost
set bit int a = 0, b = 0; for (int n : nums) { // separate into two groups if ((n & bit)
!= 0) a ^= n; else b ^= n; } // a and b are the two unique numbers
```

Q: Segment Tree vs Fenwick Tree

Feature	Segment Tree	Fenwick Tree (BIT)
Operations	Range query + Range update	Range query + Point update
Build time	O(n)	O(n log n)
Query time	O(log n)	O(log n)
Update time	O(log n)	O(log n)
Space	O(4n)	O(n)
Implementation	More complex	Very simple
Use when	Need range updates (lazy)	Simple prefix sum queries

Q: Longest Common Subsequence (DP)

LCS finds the length of the longest sequence present in both strings in the same order (not necessarily contiguous). Classic DP problem.

```
// dp[i][j] = LCS of s1[0..i-1] and s2[0..j-1] public int lcs(String s1, String s2) { int
m = s1.length(), n = s2.length(); int[][] dp = new int[m+1][n+1]; for (int i = 1; i <= m;
i++) for (int j = 1; j <= n; j++) if (s1.charAt(i-1) == s2.charAt(j-1)) dp[i][j] = 1 +
dp[i-1][j-1]; // chars match else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]); // max of
exclusions return dp[m][n]; } // Time: O(mn) Space: O(mn)
```

Q: Top K Frequent Elements

Use a HashMap to count frequencies, then a Min-Heap of size K to track the top K. Or use Bucket Sort for O(n) time.

```
// Using Min-Heap - O(n log k) public int[] topKFrequent(int[] nums, int k) { Map freq =
new HashMap<>(); for (int n : nums) freq.merge(n, 1, Integer::sum); PriorityQueue pq =
new PriorityQueue<>((a,b) -> freq.get(a) - freq.get(b)); // min-heap by frequency for
(int key : freq.keySet()) { pq.offer(key); if (pq.size() > k) pq.poll(); // remove least
frequent } return pq.stream().mapToInt(x->x).toArray(); }
```

Q: Monotonic Stack usage

A Monotonic Stack maintains elements in increasing or decreasing order. It efficiently solves 'Next Greater/Smaller Element' type problems in O(n) time.

```
// Next Greater Element public int[] nextGreater(int[] nums) { int[] result = new
int[nums.length]; Arrays.fill(result, -1); Deque stack = new ArrayDeque<>(); // stores
indices for (int i = 0; i < nums.length; i++) { // pop elements smaller than current while
(!stack.isEmpty() && nums[stack.peek()] < nums[i]) result[stack.pop()] = nums[i]; //
found next greater! stack.push(i); } return result; } // Time: O(n)
```

✓ Used for: Next Greater Element, Largest Rectangle in Histogram, Daily Temperatures, Trapping Rain Water.

SYSTEM DESIGN — Core Concepts

Q: How does Load Balancing work at scale?

Load Balancing distributes incoming traffic across multiple servers to prevent any single server from being overwhelmed. It improves availability, scalability, and fault tolerance.

- Round Robin — requests distributed evenly in rotation
- Least Connections — route to server with fewest active connections
- IP Hashing — same client always routes to same server (session sticky)
- Weighted Round Robin — high-capacity servers get more requests

Layer 4 (Transport) vs Layer 7 (Application): L4 routes based on IP/TCP, L7 routes based on HTTP content (URL, headers). L7 is smarter but more expensive.

✓ *Tools: AWS ALB/NLB, Nginx, HAProxy. Use health checks to automatically remove failed servers.*

Q: SQL vs NoSQL — Key Differences

Feature	SQL (Relational)	NoSQL
Schema	Fixed schema	Dynamic / Schema-less
Scaling	Vertical (scale up)	Horizontal (scale out)
ACID	Full ACID compliance	BASE (eventual consistency)
Joins	Supports complex joins	Typically no joins
Best for	Complex queries, transactions	Unstructured data, high scale
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, DynamoDB
Use when	E-commerce, banking	Social feeds, IoT, real-time

Q: Redis Caching – How it improves performance

Redis is an in-memory key-value store. It drastically reduces database load by serving frequently accessed data from RAM instead of disk. Typical read: database ~10ms vs Redis ~0.1ms.

- Cache-Aside (Lazy loading): App checks Redis first → if miss, query DB → store in Redis → return result
- Write-Through: Every write goes to both cache and DB simultaneously
- TTL (Time-To-Live): Keys automatically expire to prevent stale data
- Use cases: Session storage, rate limiting counters, leaderboards (sorted sets), pub-sub messaging

■ *Cache invalidation is the hard part. Plan your eviction policy: LRU (Least Recently Used) is most common.*

Q: Database Sharding – When to use

Sharding horizontally partitions data across multiple databases (shards). Each shard holds a subset of data, enabling the database tier to scale beyond a single machine.

- Horizontal Key-based: $\text{shard} = \text{hash}(\text{userId}) \% \text{numShards}$. Even distribution but hard to rebalance.
- Range-based: shard by value range (A-M on shard1, N-Z on shard2). Easy range queries, uneven distribution risk.
- Directory-based: Lookup service maps keys to shards. Flexible but adds a lookup overhead.

When to shard: Single DB exceeds storage/read-write limits, query latency degrades at scale (100M+ rows).

■ *Sharding makes cross-shard joins and distributed transactions very complex. Exhaust vertical scaling and read replicas first.*

Q: CAP Theorem Explained

CAP Theorem states that a distributed system can guarantee at most TWO of three properties simultaneously:

- Consistency (C): Every read sees the most recent write
- Availability (A): Every request gets a response (not guaranteed to be latest)
- Partition Tolerance (P): System continues despite network partitions

Since network partitions always occur in distributed systems, the real choice is CP vs AP:

- CP systems (HBase, Zookeeper, MongoDB): Choose consistency over availability. Used in banking.
- AP systems (Cassandra, DynamoDB, CouchDB): Choose availability over consistency. Used in social media.

Q: Kafka & Message Queues — Key Concepts

Kafka is a distributed event streaming platform. Producers write to topics, consumers read from topics. Messages persist on disk (unlike traditional queues) enabling replay.

- Decoupling: Producer and consumer are independent — producer doesn't wait for consumer
- Buffering: Handles traffic spikes — consumer processes at its own pace
- Partitions: Each topic splits into partitions for parallel processing
- Consumer Groups: Multiple consumers share load; each partition read by one consumer in a group

✓ *Use cases: Real-time analytics, event sourcing, log aggregation, notification systems, microservice communication.*

Q: Consistent Hashing – Why Important

Standard hashing ($\text{hash}(\text{key}) \% N$) is fragile — adding/removing a server remaps almost all keys. Consistent hashing arranges servers on a virtual ring; adding/removing a server only remaps keys from that server's adjacent range ($\sim 1/N$ keys).

- Virtual nodes: Each server gets multiple positions on the ring for better load distribution
- Used in: Amazon DynamoDB, Apache Cassandra, CDN load balancing, distributed caches

✓ *Key benefit: Minimal key remapping during scaling events — critical for caches (avoids stampede on DB).*

Q: Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Single deployable unit	Independent services
Scalability	Scale whole app	Scale individual services
Development speed	Fast initially	Faster at scale (parallel teams)
Complexity	Low initially	High (distributed systems)
Fault isolation	One bug can crash all	Failures isolated per service
Data management	Single shared DB	DB per service

Use when	Small team, early stage	Large team, proven product
----------	-------------------------	----------------------------

Q: Rate Limiting in APIs

Rate limiting controls how many requests a client can make in a time window, protecting backend services from abuse and ensuring fair usage.

- Token Bucket: Tokens added at fixed rate; each request consumes one token. Allows bursts.
- Leaky Bucket: Requests queue up and process at fixed rate. Smooths bursts.
- Fixed Window Counter: Count requests per minute window. Simple but edge-case bursts possible.
- Sliding Window Log: Most accurate — track timestamps of each request in a rolling window.

✓ *Redis is commonly used to store rate limit counters with TTL. Return HTTP 429 Too Many Requests when limit exceeded.*

Q: CDN – Reducing Latency

A Content Delivery Network distributes static assets (images, CSS, JS, video) across geographically distributed edge servers (PoPs). Users are served from the nearest edge node, reducing round-trip time.

- Push CDN: Publisher pushes content to CDN upfront. Good for predictable large files.
- Pull CDN: CDN fetches from origin on first request, then caches. Good for dynamic traffic.

✓ *Examples: Cloudflare, AWS CloudFront, Akamai. Reduces origin server load by 60-80% for static content. Also provides DDoS protection.*

SYSTEM DESIGN — Design Problems

Q: Design URL Shortener (like Bit.ly)

Functional requirements: Given a long URL, generate a short alias. Redirect short URL to original. Handle ~100M URLs, 10:1 read:write ratio.

Key Components

- Encoding: Generate 6-7 char unique ID using Base62 (a-z, A-Z, 0-9). Options: MD5+truncate, or auto-increment ID encoded in Base62.
- Database: Store mapping of shortCode -> longURL. Use NoSQL (Cassandra) for scale — simple key-value lookups.
- Cache: Redis to cache hot URLs (most popular 20% serve 80% traffic — Pareto principle).
- Redirect: HTTP 301 (permanent, cached by browser) vs 302 (temporary, every request hits your server — better for analytics).
- Custom URLs: Check uniqueness before inserting; use distributed ID generator (Snowflake) to avoid collisions.

✓ Scale: Horizontal scaling of app servers + DB sharding by shortCode hash. CDN for redirect latency.

Q: Design Instagram Feed

Core challenge: Fan-out on write vs Fan-out on read. For celebrity accounts (10M followers), precomputing feeds for every follower on post is expensive.

- Push model (Fan-out on write): On post, push to all followers' feed cache. Works for average users (<1K followers).
- Pull model (Fan-out on read): Generate feed on request by querying followed users' posts. Better for celebrities.
- Hybrid: Pre-compute feeds for normal users; pull from celebrities lazily and merge.
- Storage: Metadata in Cassandra (wide column, fast writes). Media (images/videos) on S3 + CDN.
- Ranking: ML-based ranking service scores posts by recency, engagement, relevance. Async processing.

Q: Design WhatsApp Messaging System

Core challenges: Real-time delivery, message ordering, offline support, end-to-end encryption.

- Connection: WebSocket (persistent) for real-time bidirectional messaging. Fall back to long-polling.
- Message flow: Sender → Chat Server → Kafka → Recipient's device (if online) or Message Store (if offline).
- Message ordering: Use sequence numbers per conversation. Logical clock (Lamport timestamps) for distributed ordering.
- Storage: Recent messages in Redis (fast), older messages in Cassandra (partitioned by conversationId + timestamp).
- Delivery receipts: Single tick = sent, Double tick = delivered, Blue = read. Async updates via ACK mechanism.
- Group messages: Fanout — one message record, multiple delivery pointers per recipient.

Q: Design Netflix Streaming System

Key challenges: 200M+ users, 4K streaming, adaptive bitrate, global low-latency delivery.

- Transcoding: Videos uploaded to S3, transcoded into multiple resolutions (240p to 4K) and codecs (H.264, VP9) by worker fleet.
- CDN: Open Connect (Netflix's own CDN) with edge servers in ISPs. ~95% traffic served from edge.
- Adaptive Bitrate Streaming (ABR): DASH or HLS protocols split video into 2-4 second chunks. Client dynamically switches quality based on bandwidth.
- Recommendation: Apache Spark for offline ML training. Personalized homepage generated async.
- Microservices: Eureka (service discovery), Hystrix (circuit breaker), Zuul (API gateway).

Q: API Gateway – Role and Benefits

API Gateway is the single entry point for all client requests in a microservices architecture. It acts as a reverse proxy, routing requests to appropriate services.

- Request routing: Maps API endpoints to backend microservices
- Authentication/Authorization: JWT validation, OAuth — centralized, not repeated per service
- Rate limiting & throttling: Protect backends from overload
- Load balancing: Distribute across service instances
- Protocol translation: REST to gRPC, WebSocket upgrades
- Observability: Centralized logging, distributed tracing (correlation IDs)

✓ *Examples: AWS API Gateway, Kong, Netflix Zuul, Nginx. Prevents each microservice from implementing cross-cutting concerns.*

Q: Event Driven Architecture

In EDA, components communicate by producing and consuming events via a message broker. Producers don't know about consumers — loose coupling.

- Event: An immutable record of something that happened (OrderPlaced, PaymentProcessed)
- Event broker: Kafka, RabbitMQ, AWS SNS/SQS routes events to interested consumers
- Benefits: Loose coupling, independent scaling, natural audit log, easy to add new consumers
- Patterns: CQRS (separate read/write models), Event Sourcing (state as sequence of events)

■ *Challenges: Eventual consistency, debugging event flows, ordering guarantees across partitions, schema evolution.*

Q: Design Uber Ride Matching

Core challenge: Match riders to nearby drivers in real-time with minimal latency.

- Location tracking: Drivers send GPS every 4 seconds. Store in Redis Geo (geospatial index) for real-time queries.
- Matching: On ride request, query Redis GEORADIUS for drivers within 2km. Filter available drivers, rank by ETA (Google Maps API).
- Supply/Demand: Geohash divides map into cells; surge pricing calculated per cell based on driver:rider ratio.
- Trip state machine: REQUESTED → ACCEPTED → DRIVER_ARRIVING → IN_PROGRESS → COMPLETED. Persisted in Cassandra.
- Notifications: Push notifications via APN/FCM for driver/rider status updates.

Q: Distributed Transactions Handling

In microservices, a business operation may span multiple services/DBs. Traditional ACID transactions don't work across distributed systems.

- 2PC (Two-Phase Commit): Coordinator asks all participants to prepare, then commits if all agree. Blocking if coordinator fails.
- SAGA Pattern: Break transaction into local transactions; each publishes an event triggering the next. On failure, compensating transactions undo changes.
- Choreography SAGA: Services react to each other's events. Decentralized but harder to track.
- Orchestration SAGA: Central orchestrator tells each service what to do. Easier to trace but single point of coordination.

✓ *Saga is the preferred approach for microservices. Accept eventual consistency; design compensating transactions for every operation.*

Q: Horizontal vs Vertical Scaling

Aspect	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Approach	Add more CPU/RAM to existing server	Add more servers
Complexity	Simple — no code changes	Complex — distributed system
Cost	Expensive at high end	Commodity hardware, cost-effective
Limit	Hardware ceiling exists	Virtually unlimited
Downtime	Usually requires restart	Zero downtime (rolling deploy)
Single point of failure	Yes	No (redundancy built-in)
Use when	Databases (initially), legacy apps	Web servers, stateless services

Q: Design Notification System at Scale

Requirements: Send 100M+ notifications/day across push (iOS/Android), email, SMS. Support user preferences, delivery tracking, retries.

- API layer: Notification Service receives requests, validates, and pushes to Kafka topics per channel (push, email, sms).
- Workers: Channel-specific workers (PushWorker, EmailWorker, SMSWorker) consume from Kafka, call third-party providers (APN/FCM, SendGrid, Twilio).
- User preferences: Check opt-out settings from a Redis cache before sending. Respect DND (Do Not Disturb) hours.
- Retry logic: Exponential backoff for failed deliveries. Dead Letter Queue (DLQ) for permanently failed notifications.
- Delivery tracking: Update delivery status (sent/delivered/failed) in Cassandra. Expose status API for clients.
- Rate limiting per user: Prevent notification spam — max N notifications per user per hour.

✓ *Template engine: Pre-render notification templates with user-specific data. Store templates in DB with versioning.*

■ You're Interview Ready!

Java 8 • DSA • System Design

Good luck! ■